Smart contract Lottery (specific occurrences or changes of state.)

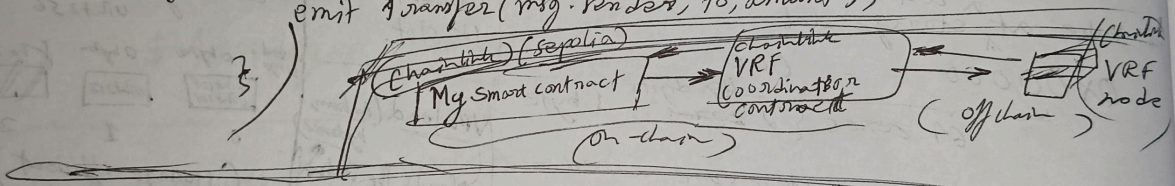
(30/4/25)

Events → log info. about the contract's execution.
→ a way for smart contracts to communicate with the outside world (clients, dApps, other contracts, etc.).
→ stored in the txn logs of the blockchain.
→ "emitted" by a fn of a contract.

- can have max. 3 indexed params (aka topics) (searchable) (can be retrieved faster than non indexed params)
- more gas efficient compared to storing large amts of data directly on the blockchain.
- non indexed params → (abi encoded)
- separate from contract storage and accessible externally.
  - Each block has its own Merkle Patricia Tries (3: Txn trie, state trie & Receipt trie (txn logs stored here))
  - (Merkle Patricia (MPT) Trie data structure)
    - (indirectly read within smart contracts.)

(Eg.) event AssetTransfer(address indexed sender, address indexed receiver, uint256 amount);

function transfer(address to, uint256 value) public {
emit Transfer(msg.sender, to, amount);
}



(31/4/25) callback fn → a fn that handles the return of an async process. → process that runs in the background without the user having to wait for it to finish.

VRF (Verifiable Random Function)
- cryptographic fn that produces a random o/p based on some i/p data.
- also, generates a proof that anyone can use to verify that the randomness was generated correctly without manipulation.

(1/2/25) FREI-PI (Function Requirements Effects-Interactions Protocol)
(also) CEI → Checks Effects Interactions pattern
(Design pattern) → secure state management pattern used to prevent reentrancy attacks.
Invariants)

Chainlink automation

- Upkeeps → automated tasks that executes predefined smart contract fn's.
- Registrar (smart contract) → handles the creation and management of upkeeps.
- Registry → smart contract that stores all registered upkeeps, tracks their conditions and triggers them when required.

(2/2/25)

vm.warp(uint256)

~~[illegible]~~

→ sets the block time (block.timestamp)

vm.roll(uint256)

→ sets block.number

(3/2/25)

Raffle.sol

→ ~~Automated~~ deploying using scripts and configuring →

- Deploy Raffle.s.sol
- HelperConfig.s.sol
- Interactions.s.sol

Mocks
- VRFCoordinatorV2_5Mock
- LinkToken (Mock)

* Players can enter raffle. (lottery)
  - → entrance fee is set. (min. fee)
* After a defined interval, raffle state → OPEN → CALCULATING
  - block.timestamp → (00:30) (say)
  - (00:00) (both) (0sec)
* In calculating state, our contract requests a random no. from chainlink VRF node through a VRF coordinator contract.

[our contract → (REQ.) coordinator contract (REQ) → VRF node
(Random no.) ← (Randomno.)]

* Now, this process needs to be automated which is done by using chainlink automation (custom logic) → checkUpkeep(bytes) → (if true) performUpkeep(bytes)

Some of the tasks are yet to be implemented.

Testing (RaffleTest.t.sol)

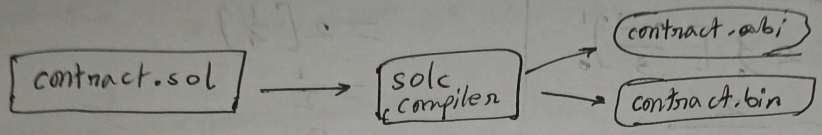
→ one test was failing because of (No new players while Raffle state is calculating) | one reverting due to some other reason than the one we wanted.

→ Invalid consumer coming from (raffle.performUpkeep("")) ;

(13/2/25)

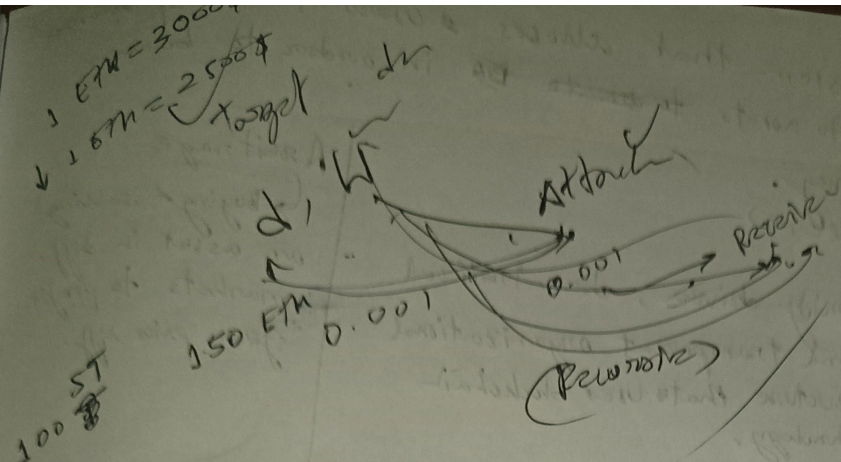
EVM opcodes, calling, etc.

↳ computation engine that handles smart contract deployments and execution.



Txn fields : (i) Nonce → txn count for the account.
(ii) Gas price (in wei)
(iii) Gas limit
(iv) To → address that the txn is sent to.
(v) Value → amt. of wei to send.
(vi) Data → what to send to the To address
(vii) V, R, S → components of txn signature

[(Eg) Fn signature ⟹ "transfer(address,uint256)"
Fn selector ⟹ 0xa9059cbb
↳ (1st 4 bytes of a fn signature)]



$10^{-5} = 1$ eth

$10^{-4} = 0.0001$

Defi (Decentralized Finance)

Stablecoins → non-volatile crypto asset (pegged → tied)
- → buying power fluctuates very little relative to rest of the market.

Fns of money → (1) store of value (depends on inflation)
(2) unit of account → used to measure the value of goods and services.
(3) Medium of exchange → used to buy goods and services.

(Types) → (I) Fiat, crypto, commodity – collateralized, algorithmic

Collateral
- → something pledged as security for repayment of a loan
- → (stable coins) asset backing the stablecoin token.

(II) (1) Relative stability (better)
→ pegged / anchored (OR) floating

(2) Stability method
→ governed (OR) Algorithmic

(3) Collateral type → Endogenous (OR) Exogenous (outside)
- Endogenous → collateral originates from within a protocol.
- Exogenous → collateral originates from outside of a protocol.

CDPs → A system that allows users to lock up various crypto assets in order to borrow money. (stablecoins)
(Collateralized debt positions)

DAO → Community-driven, decentralized and transparent organizational structure that uses blockchain technology.
(Decentralized Autonomous Organization)

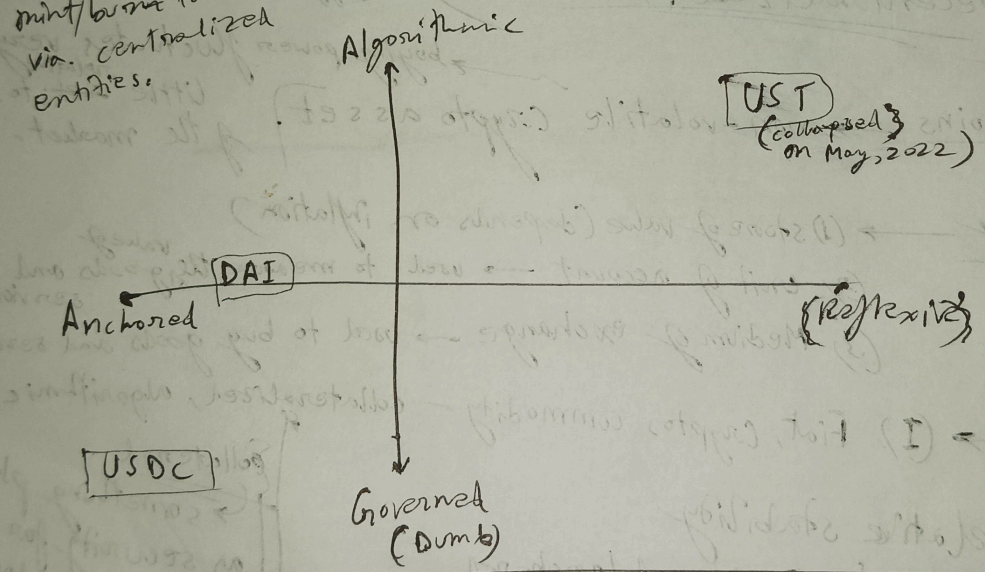
Arbitrage → Buying & selling an asset in diff. markets to profit from price diff.

Stability method of stablecoins → mechanism that keeps the value of stablecoin stable.

Spectrum

Governed (e.g., USDC) → mint/burn tokens via centralized entities.

Algorithmic (e.g., UST, DAI) → mint/burn tokens using math or autonomously executed code.



CEI ⇒ Checks, Effects, Interactions

(plummet) → fall or drop straight (↓)

Health factor → safety score for borrowed stablecoin.

$$Hy = \frac{\text{collateral} \times \text{Liquidation Threshold}}{\text{Total borrows}}$$

[$Hy < 1$ ⇒ the position may be liquidated to maintain solvency.]

$$LTV = \frac{\text{Borrowed amount}}{\text{Total collateral value}}$$

(Loan to Value ratio)

LTV ↑ ⇒ risk ↑
LTV ↓ ⇒ risk ↓

Eg, * 10 ETH deposited
* say, 1 ETH = $3000
* Total collateral value = 10 × 3000 = $30,000
* LT = 80% (= 0.8) (set by the protocol) → Max. % of collateral value that can be borrowed against stablecoins before liquidation
* Max. borrowable amt. = 30,000 × 0.8 = $24,000 (in stablecoins)
* Say, borrow = $10,000 (in stablecoins)

Then, $Hy = \frac{30000 \times 0.8}{10000} = 2.4$

$LTV = \frac{10000}{30000} = 0.33$

AST (Abstract syntax tree) → data structure ~ represents the structure of a program or code snippet.

Dynamic tests/analysis → Fuzz testing → tools → Echidna, Foundry, etc.
→ Unit testing
→ code is actually executed to determine if there's a problem.

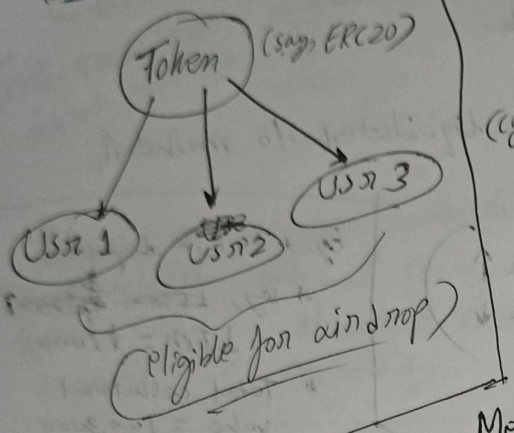
Static tests/analysis → using tools to analysis code and identify vulnerabilities. → Slither, Aderyn, etc.
(based on things like layout, ordering and syntax).

Symbolic execution
→ models each path in the code mathematically to identify if any path results in the breaking of an asserted property of the system.

Formal verification → Symbolic execution, abstract interpretation, model checking, etc.
(Tools) Manticore, Halmos, certora, etc.
→ act of proving or disproving a property of a system.
(by generating a mathematical model of the system and using mathematical proofs to identify if a property can be broken)
[special language: SMT-LIB]
(Limitations) path explosion problems

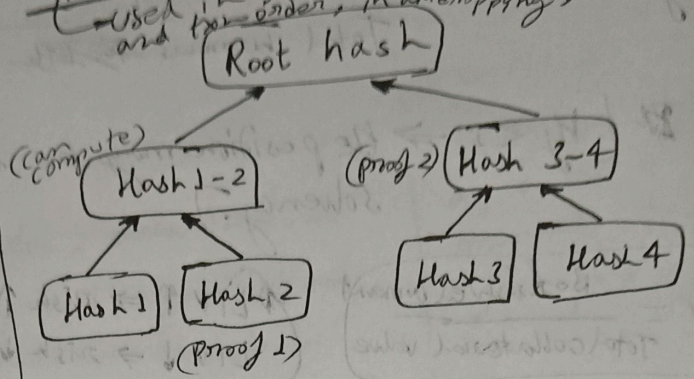
(25/2/25)

Merkle airdrop



→ (Ralph Merkle, 1979)

Merkle tree → data structure
→ used in rollups to verify state changes and transfer order, in airdropping, etc.



Merkle proof → verifies that a specific piece of data is part of a merkle tree and consists of hashes of sibling nodes present at each level of the tree.

(keccak256)

(Eg.) to prove Hash 2 is part of the given merkle tree, Hash 1 (sibling 1) at the 1st level and combined hash of Hash 3 and Hash 4 at the 2nd level can be provided as proofs.

→ Then the merkle tree verifier can reconstruct a root hash and compare it to the expected root hash. If they match, Hash 2 is confirmed to be part of the merkle tree.

---

bytes32 leaf = keccak256(bytes.concat(keccak256(abi.encode(account, amount)))

* Hashing 2 times to mitigate 2nd preimage attacks → attacker tries to find a diff. i/p that produces the same hash as an existing one

Helps identify what kind of data is being signed and prevents misuse

---

EIP-191 → defines a way to add a "header" to data before signing it.

EIP-712 → provides a way to sign complex, human-readable data in a secure and user-friendly way.

ecrecover (built-in solidity fn) → recovers the address of the signer from a signed message.

Replay attack (in ethereum) → a valid txn is maliciously repeated or "replayed" on another network or chain.

hashed message/data + private key of sender → signature (cryptographic) [using ECDSA]

r (32 byte value) | s (32 byte value) | v (1 byte value) (components of the signature)

ECDSA → Elliptic Curve Digital Signature Algorithm

Digest → hashed data we get after hashing original data.

Signatures (in ethereum)

(1) (sender) [without EIPs, simple signer]
message = "Hello, world!"

hashed message {0x ——}

signed (using ECDSA) [with private key of sender]

r, s, v

r → part of signature (x-coordinate of curve pt.)

s → another part of signature related to the message and private key

v → recovery id (helps in recovering the public key from the signature)

Sender = 0x A
receiver = 0x B

(receiver)
hashed message + v r s using [ECDSA signature recovery] [ecrecover in solidity] → generated public key of sender

If generated public key == actual public key
→ valid message

(EIP-191)

{standardizes what the signed data should look like?}

{signifies the data is a signature (not a txn)}

0x19 < 1 byte version > < version specific data > < data to sign >

(prefix)

{version that the signed data is using}

{eg: for version: 0x00, version specific data = intended validator address (EOA or contract address)}

message = "Hello, World!"

hashed message

keccak256 (abi.encodePacked (prefix, version, version specific data, application specific data (message) (hashed message)));

same process as before — v, r, s

[allowed values]

0x00 → Data with intended validator

0x01 → structured data

0x45 → personal sign messages

{a unique hash that ties the signature to a specific app-or contract}

(EIP-712) 0x19 0x01 < domainSeparator > < hashStruct(message) >

{standardizes the format of the version specific data and the data to sign}

hashStruct(eip712Domain) = keccak256(typeHash || hash(structData))

hashStruct(message)

struct eip712Domain = {
string name;
string version;
uint256 chainId;
address verifyingContract;
bytes32 salt;
}

{a unique context for the signature}

hashStruct(structData)

keccak256(typeHash || hash(structData))

(hashed) typeHash = (keccak256) "Message(string message)"

(hashed) (bytes, message(Smessage: "Hello, world!"))

(eg:) (EIP712Domain (string name, string version, bytes32 salt))

0x19 0x01 < hash of who verifies the signature and what does the verifier looks like > < hash of signed structured message, and what the schema looks like >

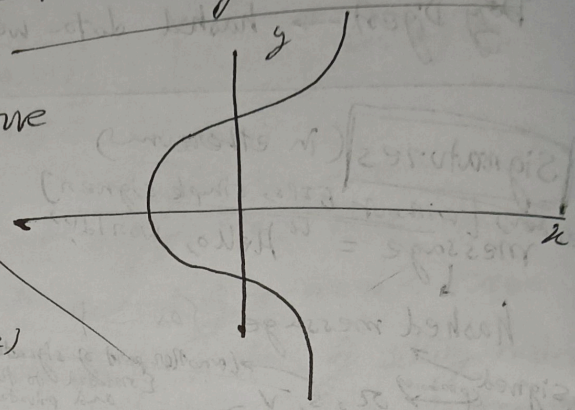
(26/2/25)

public key cryptography (also) → method of encrypting data using a pair of keys: (public key) → (private key)

asso asymmetric encryption

Ethereum address — last 20 bytes of the hash of the public key.

ECDSA (in Ethereum) → secp256k1 curve



constants → * G (Generator pt.) {const. pt. on curve}

* n (order) {prime no. generated using G} → defines the length of private key

{for 1 x, 2 y's}

R: x-pt. on the secp256k1 curve

S: proof that the signer knows the private key → {calculated using a random const. k} → {ensures signature is unique every time}

V: gives whether in the +ve or -ve (polarity) part of the curve. → {used to recover the public key from R} → {represents index of the pt. on the elliptic curve}

private key {generated} → as a random integer within the range 0 to n^{-1} (order)

public key = P · G (Generator pt.)

(private key) (modular multiplication)

Elliptic Curve Discrete Logarithm problem → Impossible to calc. P from publkey = P · G.

Signature malleability attacks

* pt. arithmetic in a finite field (?) does not support division.

EIP-4844

(27/2/25)

Upgradable smart contracts (smart contracts where functionality can be modified after deployment)

(how?) parameterize

(1) ~~parameters~~ way (not really)

(2) social migration ways

(3) Proxies

ERC-1967

→ * Implementation contract
* proxy contract (calls)
* Users (calls)
* admin → decides when to deploy a new implementation contract

* Transparent proxy
* Universal upgradable proxy (UUPS)
* Diamond pattern

Storage
slot 0
slot 1
slot 2

* Storage Clashes
* Function selector Clashes

(28/2/25)

{DAOs}

Aragon → no-code tool to create DAOs.

(1/3/25) (Security)

* Solcurity
* simple-security ~~-toolkit~~ toolkit

Manual review
Tooling
* test suites
* Static analysis

How signatures created?

hash(message) + private key } done using ECDSA algorithm

Step-1: Message → hashed message

Step-2: securely rnd. no. k (nonce) is generated.

Step-3: Rd. no. R is calc.

$R = k \cdot G$ (generator pt.)

$R = (x, y)$

$r = x \bmod n$ (order) (modulo operator)

(r of RS)

Step-4: s is calc. using k, hashed message, private key, r and n.

Step-5: v then defines which part of y axis to use. (Axe or -Ve)

(verification)

ECDSA verification algo.

i/p: signed message, signature, public key of sender.

o/p: bool (whether signature is valid or not)

$S1 = s^{n-2} \pmod n$

$R' = (h * S1) * G = (r * S1) * \text{pubkey}$

$R' = (x, y)$

$r' = x \bmod n$

$r' == r$?